

Crash-Consistent Checkpointing for Exactly-Once Agent Effects: A Write-Ahead Claim Log for LLM Agent Harnesses

Mohammed Kamran Syed

Abstract—LLM agent harnesses checkpoint *trajectories*, not *effect commitments*. When the harness crashes between a tool-effect commit and the checkpoint write, the resumed agent re-fires the tool, producing a duplicate side effect—the “double refund” window. Idempotency keys alone cannot close this window in general: content-hash keys break when the recovered agent rewords arguments, and deterministic (*workflow, step, action*) keys break when replanning shifts step indices. We prove this fragility formally for derivations that actually change across the recovery replan (Theorem 1) and propose a *write-ahead effect-claim log*: the claim and its key are logged durably *before* the tool is invoked, checkpoints are fenced by the log, and recovery reconciles from the log first under a fencing epoch against stale writers. Across a scripted sandbox (1,500 episodes/condition), two-model LLM validation (140 episodes; 210 with the tight-prompt ablation), component ablations, and a production-shaped LangGraph deployment with real SIGKILL injection, the protocol commits *zero duplicate effects* in every campaign (one-sided exact 95% upper bounds: 4.87% at $n=60$, 0.20% at $n=1,500$), where the baseline duplicates at 0.77–0.83 and idempotency keys at 0.21–0.37. Completion (every accepted effect committed exactly once) reaches 1.000 in the sandbox, the LangGraph campaigns, and LLM validation with gpt-4.1-mini (70/70) and with a tightened recovery prompt for gpt-4o-mini (70/70); under the loose prompt gpt-4o-mini completes 66/70, the 4 misses being prompt-fixable model re-emission loops, never duplicate commits. Claim-log write latency is p50 20.3 ms per step (fsync’d Claim/Commit records).

Index Terms—Fault tolerance, crash consistency, exactly-once semantics, write-ahead logging, LLM agents, checkpoint-restart, idempotency, fencing.

1 INTRODUCTION

A customer support agent issues a \$212 refund, then its harness process is OOM-killed. The harness restarts, loads its last checkpoint—which was written *before* the refund tool returned—and, seeing no record of the refund, issues it again. The customer is refunded twice. The ledger shows two commits; the checkpoint shows one intent. Nobody is at fault except the architecture: the harness checkpointed the agent’s *trajectory* (messages, tool-call records) but not the world’s *commitments*.

This is the normal operating picture for modern agent harnesses. LangGraph, CrewAI, AutoGen, and the OpenAI Agents SDK all persist enough state to resume a conversation after a crash. None, as of this writing, provides crash-consistent exactly-once semantics for *external side effects*: work performed between the last checkpoint and a crash “can run again,” and LangGraph has been measured re-executing durably recorded work after real process kills [4]. As agents move from demos to production—handling money, sending customer-facing email, mutating systems of record—the “double refund” window stops being an edge case and becomes a correctness bug with financial consequences.

The database community solved the analogous problem decades ago with write-ahead logging (WAL): never mutate state without first durably recording the intent, so recovery can reconcile. But the agent setting breaks a load-bearing assumption of classical WAL: in a database the writer never re-synthesizes its own transaction identity after a crash, while in an agent harness the *recovered agent is an LLM* that replans—rewording arguments, inserting steps, reordering its plan. Any recovery scheme that depends on the recovered agent re-deriving the *same* effect identity is fragile whenever the re-derivation actually changes—and, as we prove in §5 for that case, the failure is structural, not an artifact of a particular key scheme.

The industry-standard remedy is idempotency keys: attach a key to each tool call so the tool can suppress repeats. Keys help—we measure duplicate rates falling from 0.77 to 0.21–0.37—but they fail exactly where LLM re-synthesis is most characteristic. A content-hash key breaks when the recovered agent rewords arguments (50% reword rate in our scripted recovery, 11% with real models); a deterministic (*workflow_id, step_index, action*) key breaks when replanning shifts step indices (30% plan-shift rate). In both cases the retry presents an unseen key, the tool commits again, and the duplicate lands. Keys-alone pushes all of recovery correctness into a value re-derived *after* the crash, by an agent whose re-synthesis the harness cannot constrain.

This paper makes three contributions.

(1) **A fault class.** We formalize the *checkpoint window*

$W(s) = (t_{\text{commit}}(s), t_{\text{ckpt}}(s))$: the open interval between a tool-effect commit and the harness checkpoint write for step s . This is a *harness-side* crash-consistency gap, distinct from the service-boundary late-commit window studied in recent agent fault-injection work [1]: there the uncertainty is *whether the service committed*; here the tool *has* committed and the uncertainty is *whether the harness recorded it*. The distinction determines the fix.

(2) A mechanism with a scoped necessity argument.

We propose the *write-ahead effect-claim log*: before invoking a tool, the harness durably logs a CLAIM carrying the effect’s identity and key; checkpoints may advance only past logged COMMIT records (*log-fenced checkpoints*); recovery reconciles from the log *first*, replaying uncommitted claims with their original keys, under a fencing epoch that rejects stale pre-crash writers. We prove (Theorem 1) that when a retry-time key derivation actually changes across an admissible recovery replan of the same logical effect, a keys-only receiver can commit both attempts—and that shifted keys colliding with another operation’s committed key cause erroneous suppression instead; E2a/E2b map the two failure modes empirically. Under the recovery-information model of §5, durable write-ahead identity information is the necessary ingredient and our claim log is a sufficient mechanism. Theorem 2 then separates the guarantees: at-most-once commitment per durable claim identity (safety), eventual commitment of accepted claims under explicit progress assumptions, and workflow completion as a property of the planning policy, not the protocol.

(3) Measurement. We evaluate across four layers: a scripted sandbox (1,500 episodes per condition) isolating the fault; two-model LLM-driven validation (140 episodes: 70 per model), plus a 70-episode tight-prompt ablation (210 LLM episodes total) with real model re-synthesis on recovery; component ablations proving each protocol piece load-bearing against disjoint failure modes; and a production-shaped LangGraph deployment (branching supervisor, parallel fan-out, retries) with real SIGKILL injection, including a real-LLM supervisor under uniform and mixed routing. The write-ahead log achieves **0.000** duplicate-effect rate in every campaign and **1.000** exactly-once rate in all campaigns except gpt-4o-mini under the loose recovery prompt (66/70; tight prompt: 70/70), against baseline duplicate rates of 0.77–0.83 and idempotency-key rates of 0.21–0.37. Claim-log overhead is p50 20.3 ms per step—about a 1% tax on multi-second agentic steps.

The remainder of this paper is organized as follows. §2 surveys related work and positions this paper against the nearest neighbors. §3 formalizes the checkpoint window and walks through a concrete failure. §4 presents the protocol, its record format, and the reconciliation algorithm. §5 gives the fault model, both theorems, and honest limits. §6 and §7 report the evaluation and ablations; §8 discusses the key-derivation boundary our results reveal; §9 states the limitations; §10 concludes.

2 BACKGROUND AND RELATED WORK

2.1 Idempotency keys for agent tools

The closest empirical work is Li’s *Where Does Exactly-Once Live?* [1] (whose sandbox is named Limbo), which applies

idempotency keys to agent tool calls and measures duplicate reduction from 28% to 4% across 12 injected fault modes. Its fault taxonomy is service-boundary throughout: the late-commit window it studies is the interval between the agent’s dispatch and the service’s commit becoming visible—the uncertainty is *whether the service committed*. Its text contains zero mentions of “crash” or “write-ahead”; the harness-side checkpoint window is untested there. Our work is complementary: Li measures what keys achieve at the service boundary; we show where keys fail at the harness boundary and what structure is needed instead. We adopt the same tool-contract idealization (atomic check-and-commit) as shared ground. Its residual 4% lives at the *service* boundary (late commits, in-flight ambiguity), a different fault class from our harness-side commit→checkpoint window; we do not claim to explain it. What transfers is the lesson: wherever recovery correctness depends on re-deriving an identity after the failure, some residual remains, and only durable records close it.

2.2 Crash and rollback in agent frameworks

Three recent papers are adjacent must-cites. Khan’s *Resume Means Resume* [4] gives a machine-checked (TLA+/TLAPS) resume contract for workflow persistence layers and independently measures LangGraph re-executing durably recorded work after real SIGKILL—direct confirmation of the baseline fault our sandbox reproduces (0.80 duplicate rate). Where Khan specifies what resume *must mean*, we show *how to achieve* exactly-once for the commit→checkpoint window and measure it: contract and mechanism, complementary. Zheng et al. [5] formalize exact checking of execution edits (checkpoint/fork/restore/merge) in Lean, noting that unsafe edits can “authorize the same tool action twice”; their solution is a decision procedure for edit safety, not a crash-consistency mechanism. *Safe to Resume?* [6] documents five rollback failure modes including unrecorded external effects with a duplicate-payment demonstration on LangGraph, under an adversarial framing. ACRFence [2] is the closest adjacent mechanism: it *records irreversible tool effects* and enforces replay-or-fork semantics upon restore, overlapping with our durable effect records in purpose if not in form. The differences are the threat model—semantic rollback *attacks* (Action Replay, Authority Resurrection) versus our benign crash-stop fault—and the mechanism: ACRFence constrains what a *restored* agent may re-issue, while our write-ahead claim log plus reconciliation-first recovery and epoch fencing make re-issuance unnecessary in the first place. Our fault model is deliberately benign (crash-stop, chaos/HPC checkpoint-restart tradition); there is no adversary in this paper. AgentRewind [3] records aligned checkpoints of agent context and controlled environment for long-horizon recovery, but explicitly cannot undo external effects: the opposite direction from ours.

2.3 Logs for agents

LogAct [7] is the closest published neighbor in spirit: it structures agents as state machines playing a shared, append-only AgentBus and explicitly discusses crash recovery via the bus as a write-ahead log, including fencing

of stale (old-driver) writers after a crash. Its paper states its primary focus as *single-agent* safety and fault tolerance; the bus mediates among deconstructed agent roles with pre-execution voting and semantic (LLM-driven) recovery. What distinguishes our work is the *atomic receiver contract* paired with a *harness-crash recovery guarantee*: LogAct does not specify an idempotent atomic check-and-commit at the tool boundary, nor does it prove or measure exactly-once effect semantics under crash—its evaluation covers recovery correctness, token savings, and attack blocking, not the commit→checkpoint window. Where LogAct’s bus mediates among deconstructed agent roles, our log mediates between one crash-prone harness and its tools. The closest formal treatment of the checkpoint crash window we can identify is Khan’s machine-checked conformance contract for checkpoint, interrupt, and resume semantics in workflow persistence layers [4]: a TLA+/TLAPS specification of what resume must mean, with independent measurements confirming that LangGraph re-executes durably recorded work after real process kills—but without agents, LLMs, tools that keep their own records, or a mechanism. We cite it as the closest formal neighbor and distinguish our constructive, measured contribution: Khan specifies the contract; we supply the mechanism and the measurement.

2.4 Durable execution and sagas

The systems lineage is write-ahead logging and durable execution. The canonical WAL treatment is ARIES [10]: never mutate state without first durably recording the intent, so recovery can reconcile—the discipline our claim log adapts. Classical WAL assumes a writer that never re-synthesizes its own operation identity after a crash; the log record and the retry trivially coincide. RIFL [9] is the closest classical analogue to our receiver contract: it converts at-least-once RPCs to exactly-once by durably recording completed-call results and returning the original result on retry, migrating that metadata with the data across reconfiguration. Our tool-side idempotent receiver (§4) follows the same idea at the harness/tool boundary, though our current contract returns status only on duplicates—original-result return is specified future work (§9); what is new is the *harness-side* half of the problem—the recovered agent is an LLM that re-plans, rewords, and re-indexes, so the retry may never present the recorded identity at all. No receiver-side mechanism alone (RIFL included) can suppress a retry it cannot recognize; the claim log’s FIND_BY_IDENTITY reconciliation exists precisely to re-attach the original identity before the tool is consulted. Modern durable execution (Temporal, DBOS-style) gives exactly-once workflow execution via durable logs and replay. Crucially, Temporal does *not* require the LLM’s nondeterministic outputs to be re-derived: LLM invocations run as Activities *outside* the deterministic replay path, and their results are recorded durably, so replay reuses the recorded decision rather than re-invoking the model. What our mechanism adds is the *harness-side* commit→checkpoint window for external side effects: even with activity results persisted, a crash between the tool’s real-world commit and the harness’s checkpoint of that commit leaves the harness unaware the effect landed, and the recovered agent—re-planning, not replaying—may re-issue it under a re-derived

identity. The claim log closes exactly this window: durable pre-commit identity records plus reconciliation-first recovery, at the granularity of individual tool effects rather than workflow activities. The saga pattern [11] addresses multi-step transaction compensation (semantic atomicity across services), a different correctness property from exactly-once of individual effects; our protocol composes with saga-style compensation (the claim log is a natural saga journal) but does not require it.

In sum: no prior work combines the benign commit→checkpoint crash window as the fault, a write-ahead effect-claim log with reconciliation-first recovery as the fix, measured zero duplicates at exactly-once 1.000, and an analysis of idempotency-key fragility under rewording and plan shift. A pre-submission literature re-check (October 2026) confirmed this against the active 2026 literature, which clusters around adversarial rollback, formal contracts and checking, sandbox checkpoint/restore, and oversight logging—all adjacent problems or different solutions.

3 THE CHECKPOINT WINDOW

3.1 Formalization

Consider an agent harness H executing a workflow of effectful steps against external tools T (durably recording every committed effect). For step s , order the relevant events: $t_{\text{claim}}(s) < t_{\text{call}}(s) \leq t_{\text{commit}}(s) < t_{\text{mark}}(s) < t_{\text{ckpt}}(s)$: the effect is claimed, the tool invoked, the effect committed (the real world changes), the commit recorded, the checkpoint advanced.

Definition 1 (Checkpoint window). $W(s) = (t_{\text{commit}}(s), t_{\text{ckpt}}(s))$: the open interval between the tool-effect commit and the harness checkpoint write for step s .

A crash-stop failure of H at time t_c lands, for the step in flight, in one of three places: *pre-step* ($t_c < t_{\text{commit}}(s)$, safe—the effect never committed); *in-window* ($t_c \in W(s)$, the fault under study—the effect committed but the harness does not know it); or *post-checkpoint* ($t_c \geq t_{\text{ckpt}}(s)$, safe—commit recorded and checkpointed). Figure 1 shows the event timeline. Recovery from the last durable checkpoint replays from before s ; the agent re-fires the tool; the effect duplicates.

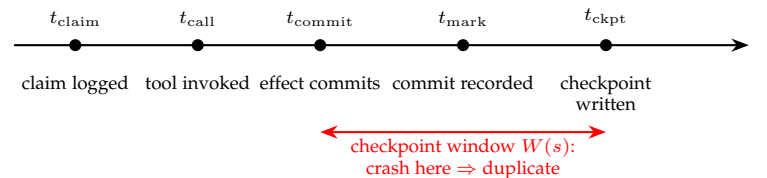


Fig. 1. Event timeline for an effectful step. A crash landing in $W(s)$ —after the real-world commit but before the checkpoint write—is the fault under study.

Two properties make this window specifically an *agent* problem. First, the recovered agent re-synthesizes its plan with an LLM: arguments get reworded, steps get inserted or reordered, so any recovery scheme depending on re-deriving the *same* effect identity is fragile by construction.

TABLE 1

Positioning: fault model, durable records, identity handling, receiver assumptions, fencing, guarantee, and evaluation across adjacent work. “Harness-side” = the commit→checkpoint window $W(s)$; “service-side” = the dispatch→commit visibility window.

Work	Fault model	Durable records	Identity handling	Receiver assumptions	Fencing	Guarantee	Evaluation
This paper	Benign crash-stop, harness-side $W(s)$	Per-harness write-ahead effect-claim log; log-fenced checkpoints	Semantic (<i>action</i> , <i>target</i> , <i>occurrence</i>); original keys replayed, never re-derived	Idempotent atomic check-and-commit; DUPLICATE_SUPPRESSED status on duplicate	Epoch fencing vs. stale writers	Exactly-once per durable claim identity; 0 duplicates measured	1,500 eps/cond sandbox; 210 LLM eps; 60 eps/cond LangGraph + SIGKILL; ablations; E1–E5
Li (“Limbo”) [1]	Service-side: timeouts, late commits, redelivery	Ledger of committed effects; per-service key sets	Keys attached by harness guard; re-synthesis out of scope	Optional idempotency keys; eventually consistent	None	Keys cut duplicates 28%→4%; Prop. 2 gives conditional exactly-once for same-key reuse only	25,930 eps; 9 models; 12 fault modes
ACRFence [2]	<i>Adversarial</i> rollback (Action Replay, Authority Resurrection)	Record of irreversible tool effects	Re-synthesized requests treated as new (the attack)	Server-side replay-or-fork enforcement	Implicit in replay-or-fork	Mitigation of semantic rollback attacks	PoC experiments
LogAct [7]	Crash-stop (agent or environment); primary focus <i>single-agent</i>	Shared append-only AgentBus (WAL discipline); old-driver fencing	State-machine replay from bus; semantic (LLM) recovery	Voters approve pre-execution; executors typed	Old-driver fencing on bus	Consistent recovery; attack blocking	Recovery correctness, token savings, attack blocking
AgentRewind [3]	Errors propagating through context/environment	Aligned checkpoints of agent context + controlled environment	Resume with prior-attempt information	Controlled environment only	None	Improved task success / check-list progress	MettleBench long-horizon tasks
Safe to Resume? [6]	<i>Adversarial</i> rollback; 5 modes incl. unrecorded effects	Varies by framework (the gap under study)	Nondeterministic replay characterized	Framework-dependent	None (failure mode)	None—documents failures; double-payment demo	5 frameworks; 3 end-to-end attacks
Khan [4]	Crash-stop resume	Workflow persistence layer	N/A (contract level)	N/A	N/A	Machine-checked (TLA+/TLAPS) resume contract	LangGraph re-execution measurement
RIFL [9]	Server crash, benign	Durable completed-RPC result records	Client-supplied request IDs; retry recognized by ID	Returns saved result on duplicate; lease-GC’d	N/A (no stale writers)	Linearizability (exactly-once RPC)	RAMCloud: <4% overhead
ARIES [10]	System crash, benign	WAL; dirty-table; checkpoints	LSN-identified log records; no re-synthesis	Page-level redo/undo	N/A	Atomicity + durability	Analytical; systems at IBM
Sagas [11]	Step failure, benign	Saga log; compensating transactions	Saga/step IDs; no re-synthesis	Compensating actions per step	N/A	Semantic atomicity (all-or-compensated)	Analytical

Second, classical write-ahead logging assumes the writer never re-synthesizes its own transaction identity after a crash; the agent harness violates that assumption routinely. The fix must therefore make recovery *independent* of the agent’s post-crash re-derivation.

4 DESIGN: THE WRITE-AHEAD EFFECT-CLAIM PROTOCOL

4.1 Principals and record format

Principals. H : the crash-prone agent harness process. T : the external tool/service, durable across H crashes, and an *atomic idempotent receiver*: $T.\text{call}(k, e, \text{args})$ atomically checks whether key k was seen and commits the effect iff not, returning COMMITTED or DUPLICATE_SUPPRESSED (status only; the original result payload is not returned —

§9). It keeps a durable per-workflow key set and a per-workflow fencing epoch max_epoch , rejecting calls with $e < \text{max_epoch}$. Fence acquisition is max -monotonic at T : the durable arbiter of the current epoch is T , not the harness’s local epoch file. L : the write-ahead effect-claim log, durable (fsync’d append-only JSONL in our implementation, ~376 bytes per step). C : the checkpoint (last fully-recorded step index), durable.

Each claim record carries the fields in Table 2: the workflow and step identity, the key fixed at claim time, the arguments as invoked, the fencing epoch, and the record type. The key need only be deterministic *at claim time*—post-crash re-derivability is irrelevant, because the log supplies the key (see Remark 1).

Claim discipline. H never invokes T without a prior durable $\text{CLAIM}(\text{wid}, s, k(s), \text{args}, \text{epoch})$ in L . This is

TABLE 2
Claim-log record format. The logical identity is
(*action*, *target*, *occurrence*); see Definition 2.

Field	Purpose
wid	Workflow identifier
step	Step index within the workflow
key	Idempotency key, fixed at claim time
action, target, occurrence	Logical claim identity (Definition 2)
args	Arguments as invoked
epoch	Fencing epoch of the claiming generation
type	CLAIM or COMMIT

harness-enforced: the harness refuses the tool call otherwise. In our LLM validation the discipline held at 98.3% of steps (100% of episodes); violations degrade to *refused calls*—the safe direction (missing, never duplicates).

Where occurrence comes from. The harness assigns *occurrence* at claim time as a per-(*action*, *target*) sequence counter: the first claim for a pair gets 0, the next intentional re-claim of the same pair gets 1, and so on. It is stored durably in the claim record (Table 2), so it survives any crash exactly like the key does. On recovery, the agent’s retry names the occurrence it intends (e.g., “the second authorized refund to this customer”); the skip lookup matches the full triple (*action*, *target*, *occurrence*) against *L*. A retry that cannot name its occurrence is a deployment whose identity function is underspecified (L4)—the protocol cannot disambiguate what the deployment itself leaves ambiguous.

Claim creation is create-or-reuse. Before appending a CLAIM, *H* looks up the full (*action*, *target*, *occurrence*) triple in *L*. If a durable CLAIM with that identity exists, *H* reuses its key and issues *no* new claim—this is the path a post-crash retry takes. Otherwise *H* mints a previously-unused key, appends the CLAIM durably, and only then invokes the tool. The lookup-then-append ordering is what preserves Definition 3’s invariant across crashes: a crash between the lookup and the append can leave at most a durable claim whose key a later attempt will *reuse*—never two keys bound to one identity—because claim discipline forbids any tool call without a durable claim, so a key that never reached *L* could never have reached *T*. The occurrence counter is a pure function of the durable claims (the count of claims for the (*action*, *target*) pair), so no separate counter state exists to desynchronize across a crash. A scripted trace exercises this end to end (`results_e7_refunds.json`): two separately authorized refunds to the same target commit (2 commits, 2 distinct keys); a crash follows; retries of each refund hit the reuse path and are suppressed (0 new commits); a third authorized refund commits as a new identity (3 commits total). The invariant audit confirms one key per identity and distinct keys per distinct identity.

4.2 Normal and recovery paths

Normal path for an effectful step *s*:

- 1) *H* runs *create-or-reuse*: look up (*action*, *target*, *occurrence*) in *L*; reuse the existing key if a CLAIM is durable, else mint a fresh key and append CLAIM(*s*) to *L* (*write-ahead*).
- 2) *H* invokes *T.call*(*k*(*s*), *epoch*, *args*).

Algorithm 1 Reconciliation-first recovery

```

1: epoch  $\leftarrow$  CAST(epoch + 1) {atomic fence acquisition at
   T; single-recovery-writer assumed in our implementa-
   tion (§4)}
2: for each CLAIM(s)  $\in$  L without matching COMMIT do
3:   T.call(k(s), epoch, args(s)) {original key from L}
4:   append COMMIT(s) on success
5: end for
6: advance C past all COMMITTED claims {log-fenced}
7: resume agent from C; check post-recovery steps against
   L by (action, target, occurrence) before issuing new
   claims {semantic skip}

```

- 3) *H* appends COMMIT(*s*) to *L*, then advances *C* past *s*. *C* may advance past *s* **only if** COMMIT(*s*) $\in$ *L* (*log-fenced checkpoint*).

Recovery path (*H* restarts; only *L*, *C*, *T* survive) is given as Algorithm 1.

A torn trailing record in *L* is skipped on load (standard WAL tail discipline); a torn non-trailing record is treated as corruption. A torn COMMIT is safe: the claim replays with its original key and *T* suppresses it.

Three design points deserve emphasis. Figure 2 summarizes the architecture. First, reconciliation uses the claim’s *original* key from *L*—the agent is never required to re-derive a key, which is precisely where keys-alone fails (§5). The tool’s idempotent receiver dedups if the pre-crash call committed, executes if it never arrived: exactly-once either way. Second, the fencing epoch closes the *zombie writer* hole: a pre-crash in-flight call arriving after recovery bears a stale epoch and is rejected, so no phantom commit can slip in after reconciliation. This holds under the crash-stop fault model with a single recovery writer per crash: epoch acquisition is read-increment-write on the harness side and atomic only via *T*’s *max-monotonic* fence; two concurrent recoveries could share an epoch (see §9). Third, log-fenced checkpoints preserve the protocol’s core invariant—*C* never runs ahead of *L*—so the checkpoint window *W*(*s*) is the *only* place an in-window crash can land, and reconciliation covers exactly that window.

The semantic skip in line 7 of Algorithm 1 handles the LLM re-synthesis problem: post-recovery steps are checked against *L* by (*action*, *target*, *occurrence*) before any new claim issues, so reworded or re-indexed retries of already-claimed effects are skipped rather than re-executed, while legitimately distinct effects sharing an action and target (different occurrence) still execute.

Parallel execution. Fan-out uses one durable file per settled branch rather than a single checkpoint record, so concurrent workers never race on a read-modify-write checkpoint; the completion frontier is the *set* of settled branches and tolerates gaps. Recovery recomputes the frontier from *L* under the log-fenced rule: a branch is settled only if *every* one of its effects has a matching COMMIT. The protocol currently assumes disjoint effect ownership across parallel workers: two workers concurrently claiming the same (*action*, *target*, *occurrence*) are not serialized by the log, and the receiver dedups by key, not by identity (see §9).

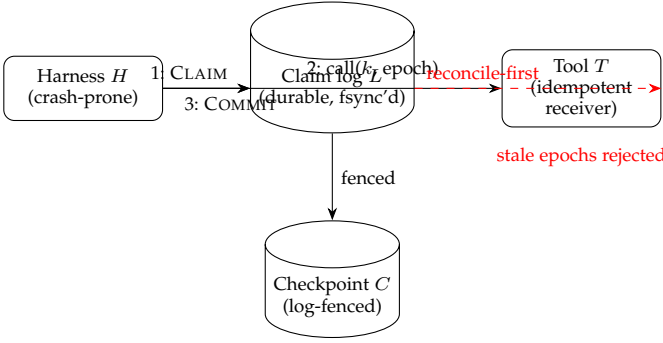


Fig. 2. Protocol architecture. The harness claims before invoking (1–2), commits after (3); checkpoints advance only past logged commits. Recovery reconciles from the log first (dashed), replaying original keys under a fresh fencing epoch.

4.3 Implementation notes

Our reference implementation uses an fsync’d append-only JSONL claim log (~376 bytes/step) and atomic checkpoint writes; the tool side is a separate-process HTTP server implementing atomic check-and-commit with per-workflow seen-key sets and fencing epochs, plus an append-only ledger that serves as ground truth for scoring. The LangGraph port (§6) reuses the same record shape and discipline inside a real StateGraph, with per-branch claims for fan-out. Crash injection is a real SIGKILL from an external watchdog process that observes progress markers only—it shares no state with the agent. Total API spend across all LLM campaigns was \$0.114 (\$0.095 for the three 70-episode validation runs, \$0.019 for the two LangGraph supervisor campaigns).

5 FORMAL ANALYSIS

5.1 Fault model

Crash-stop failure of H only (benign faults, in the chaos/HPC checkpoint-restart tradition); L , C , T persist; T itself does not crash; no Byzantine behavior. One crash per episode in our evaluation (the single-crash model standard in checkpoint-restart studies).

5.2 Necessity: keys alone cannot suffice

Theorem 1 (Retry-time key fragility). *Let K be an idempotency-key derivation computed by the recovered agent at retry time. If two admissible attempts of the same logical effect produce different keys $k \neq k'$ —because the derivation reads inputs that are mutable under re-synthesis (rewordable argument text, shiftable plan positions) and the recovery replan actually changes them—and neither key was previously used for another operation, then an atomic receiver that deduplicates only by key can commit both attempts. A keys-only retry protocol therefore cannot guarantee duplicate suppression across all admissible recovery replans under in-window crashes.*

Proof sketch. Fix a step s committed in-window under $k = K$ (pre-crash inputs). The non-invariance condition is load-bearing: mutable inputs alone do not break a derivation—a derivation that canonicalizes reworded text or ignores

plan positions never satisfies $k' \neq k$ and is outside the theorem. Two admissible recoveries exhibit the two failure modes when the derivation *does* change. (i) *Re-derivation* → *duplication* (E2b). Content-hash keys, $K = H(\text{action}, \text{args})$: the recovered agent rewords the arguments to $a' \neq a$ (admissible), so the retry presents $k' = H(\text{action}, a') \neq k$, unseen by T , which commits again—a duplicate. Empirical witness: recovery-side rewording broke content-hash keys on 46/60 episodes (92 excess commits, 0 missing effects); the WAL held 60/60 via canonicalized identity lookup. (ii) *Shift* → *duplication and collision* → *omission* (E2a). Deterministic position keys, $K = (\text{wid}, i, \text{action})$: the recovered agent rotates branch indices on replan (admissible). A retried logical step carrying $i' \neq i$ presents $k' = (\text{wid}, i', \text{action}) \neq k$ —a duplicate. Separately, a shifted retry whose new key collides with an *already-committed* key of a different operation is wrongly suppressed—a lost effect, not a duplicate. Empirical witness: positional shift broke deterministic keys on 46/60 episodes with 0.77 duplicates and 0.77 missing effects per episode; the WAL held 60/60 via `find_by_identity` key reuse. E2a is therefore a collision→omission witness (with accompanying duplicates), E2b a pure re-derivation→duplication witness—different failure modes, reported separately. The failure is a property of the *class*—retry-time derivations whose outputs actually change across an admissible replan—not of the two schemes. The harness cannot constrain re-synthesis inputs after the crash, so—when the retry-time derivation actually changes across an admissible retry (the theorem’s non-invariance condition)—no keys-only protocol can rule out the $k' \neq k$ case in advance. $\square$

Remark 1 (Scope). *The theorem is silent on keys that are not retry-time derivations: durable business-operation IDs assigned by the tool or domain and passed through unchanged lie outside its domain and may suffice. This is exactly the boundary our evaluation maps: under stable identities deterministic keys tie the WAL (v2: 0.0000/1.0000 both; E1 native persistence 60/60); under identity shift they fail (E2a/E2b).*

Remark 2 (Scoped identity-information claim). *Within the recovery-information model of this section—where the recovered agent may re-synthesize any retry-time input and no durable business identifier survives the crash outside the harness’s own records—the source of truth for effect identity must be fixed before the crash: a durable write-ahead identity record binding the effect’s logical identity to its key, not re-derived after it. Durable write-ahead identity information is the necessary ingredient under this model; the append-only claim log is a sufficient mechanism, not the unique one. Where a stable domain identifier does survive recovery, keys alone suffice and no new record is needed (E1: native persistence ties the WAL at 60/60). This remark states a scoped modeling claim, not a universal necessity theorem.*

Empirical counterpart. Scripted sandbox, 1,500 episodes/condition, in-window crash fraction 0.75: duplicate-effect rates—baseline (no keys) 0.7687, content-hash keys 0.3667, deterministic keys 0.2113, write-ahead claim log **0.0000**. LangGraph production port with real SIGKILL: under *stable* identities deterministic keys tie the WAL (0.0000/1.0000 both; E1 native persistence likewise 60/60)—keys suffice there. Under *identity shift* across recovery

(E2a positional, E2b content rewording), deterministic keys break on 46/60 episodes each while the WAL holds 60/60. The measured gap is the price of re-derivation fragility, paid exactly where Theorem 1 says it is due.

Definition 2 (Logical effect identity). A logical effect identity is a function $id : \mathcal{E} \rightarrow \mathcal{I}$ from tool effects to an identity space, supplied per deployment, satisfying: (i) *stability*— id is invariant under argument rewording and plan repositioning (business meaning, not surface form); (ii) *discrimination*—distinct business intents map to distinct identities even when (action, target) coincide (e.g., two separately authorized refunds to the same recipient differ by an occurrence/authorization component); (iii) *retry equivalence*—a post-recovery retry r of a claimed effect c is equivalent iff $id(r) = id(c)$; equivalent retries must be suppressed, non-equivalent effects must both commit. Default instantiation: $id = (action, target, occurrence)$. Misidentification breaks the protocol in either direction; the identity function is a deployment modeling assumption, not a theorem. All exactly-once claims below are per durable claim identity under the deployment's id .

5.3 Correctness: the protocol guarantees exactly-once

Definition 3 (Key–identity invariant). The protocol maintains: (i) every logical identity has exactly one durable key, fixed in its CLAIM record; (ii) all equivalent attempts of that identity reuse that key (reconciliation replays the original key from L ; the skip lookup matches on the full (action, target, occurrence) triple); (iii) distinct accepted identities receive distinct keys. The receiver deduplicates by key; the theorem guarantees uniqueness by logical identity; this invariant is what connects them—a duplicate key implies an equivalent attempt, and a distinct identity implies a distinct key.

Theorem 2 (At-most-once commitment). Under (A1) atomic idempotent T , (A2) durable L and C , (A3) crash-stop H only, (A4) epoch fencing, (A5) claim discipline, (A6) disjoint effect ownership across concurrent workers (two workers never concurrently claim the same logical identity), and (A7) a single recovery writer per crash—for **any** crash time t_c and **any** post-recovery agent behavior that assigns the correct logical identity (occurrence) to each attempt, no durable claim identity (per the deployment's id) commits more than one tool effect. The guarantee is conditional on correct logical-identity assignment: an agent that mislabels the occurrence (calling the original refund “occurrence 1” instead of “occurrence 0”) presents a new identity the log cannot link to the original business intent, and logging alone cannot recognize the duplicate intent (Definition 2).

Theorem 3 (Eventual commitment of accepted claims). Under (A1)–(A7) plus progress assumptions (P1) T eventually processes every call it accepts, (P2) the harness eventually runs recovery to completion, (P3) fencing epochs are acquired atomically—every CLAIM durably accepted into L is eventually committed by T . Accepted claims are at-least-once; with Theorem 2, exactly-once per claim identity.

Remark 3 (Workflow completion is policy-dependent). The protocol does **not** guarantee the agent emits the remaining claims of a workflow; completion is a property of the planning/recovery policy, not the protocol. The measured 66/70 completion (94.29%, gpt-4o-mini; the 4 misses were model re-emission loops, a

supervisor-policy behavior) is an empirical completion rate under a specific policy, not a liveness proof. Safety held on all 70 episodes including the 4 incomplete ones: 0 duplicates.

Proof sketch for at-most-once. Case analysis on the crash landing for each step s , split at every durability boundary. Case 1—before CLAIM durable ($t_c < t_{claim}(s)$): no durable trace exists; recovery treats s as never attempted and a fresh CLAIM with the claim-time key commits it once. Case 2—CLAIM durable, call never arrived ($t_c \in [t_{claim}(s), t_{call}(s))$): reconciliation replays with the original key; T has never seen it and commits. Case 3—call dispatched, COMMIT not yet durable ($t_c \in [t_{call}(s), t_{mark}(s))$): two sub-cases. (3a) The call arrived at T : (A1) atomicity supplies a linearization point within the call's processing, not immediate commitment on arrival. Either the atomic check-and-commit has already linearized (effect committed; the replayed key is then found and suppressed) or it has not yet linearized (the replayed key is the first to arrive and commits). In both cases exactly one commit results, because the key is deduplicated atomically at T ; then COMMIT is written and C advances (log-fenced). (3b) The call was dispatched but not yet arrived (in-flight on the wire when H died): T has never seen the key. Reconciliation replays with the original key and T commits it. Either way, exactly one commit. This is the true in-window sub-case. Case 4—COMMIT durable, checkpoint not yet advanced ($t_c \in [t_{mark}(s), t_{ckpt}(s))$): COMMIT(s) $\in L$ but C not advanced past s . Reconciliation finds the CLAIM+COMMIT pair and issues no replay; C advances past s . No duplicate, no loss. Case 5—post-checkpoint ($t_c \geq t_{ckpt}(s)$): COMMIT(s) $\in L$ and C advanced; reconciliation skips s and the agent resumes after C . Stale writers: any in-flight pre-crash call arriving after recovery bears $e < max_epoch$ and is rejected by (A4); no zombie commits (E4: 102/102 offending calls were this class; 0 same-key recommit attempts succeeded). Agent-behavior independence (conditional): reconciliation uses original keys from L —the agent never re-derives a key—and post-recovery steps are checked against L by logical effect identity before new claims issue, so reworded or shifted retries of claimed effects are recognized as equivalent and skipped. The argument holds for arbitrary post-recovery behavior subject to correct logical-identity assignment: an agent that correctly names the occurrence it intends cannot force a duplicate, whatever it rewords or reorders; an agent that mislabels the occurrence defeats the check, which is the stated boundary, not a protocol failure. Observed: 11 consecutive re-emissions of committed effects, all suppressed, 0 duplicates over 70 LLM-driven episodes. $\square$

Proof sketch for eventual commitment. Reconciliation replays every CLAIM without a matching COMMIT using its original key. By (P2) reconciliation runs to completion; each replayed call either finds its key at T (already committed) or commits it (P1); (P3) guarantees the replaying epoch owns the workflow. Hence every accepted claim is eventually committed. $\square$

Remark 4 (Key determinism, sharpened). The protocol requires keys to be deterministic only at claim time (so a pre-crash claim and any same-generation retry coincide). Post-crash re-derivability is irrelevant—the log supplies the key. This is

the precise point where keys-alone fails and the write-ahead log succeeds.

5.4 Assumptions and limits

(L1) Tool idealization. (A1) assumes atomic check-and-commit at T . Real tools with non-atomic or non-idempotent APIs need an adapter (e.g., a transactional outbox at the tool boundary); the theorem does not cover them. This is the same idealization as Li’s tool contract—shared ground, not a weakness unique to this work. **(L2) Log durability.** (A2) is assumed, not proved; log loss voids the guarantee—the standard WAL assumption, stated explicitly. **(L3) Safety, not liveness.** Theorem 2 is a *safety* property (no duplicate commits; reconciliation gives at-least-once of *claimed* effects). It does not prove the agent will emit remaining claims—completeness depends on agent behavior and harness recovery policy. **(L4) Semantic identity is a modeling choice.** Skip-logic keys on (*action*, *target*, *occurrence*) as the stable business identity of an effect (Definition 2); effects whose identity cannot be captured this way need a domain-specific identity function. **(L5) Fault scope.** Crash-stop H only; no Byzantine harness, no tool-side crashes, single writer per workflow per epoch (fencing enforces this). **(L6) Claim discipline is enforced, not assumed.** (A5) held at 98.3% step-level in LLM validation because the harness *refuses* tool calls without valid claims—violations degrade to refused calls (missing, never duplicates).

(L7) Mechanism vs. information. Under the recovery-information model of §5 (Remark 2), the theorems establish the necessity of *durable write-ahead identity information*—not uniqueness of the append-only log schema. A durable operation table (identity $\rightarrow$ status) written before each tool call would satisfy the same requirement; our claim log is the evaluated sufficient mechanism. Necessity claims in this paper refer to the information *under that model*, never the schema, and never universally—consistent with Remark 2’s scoped modeling claim.

6 EVALUATION

Our evaluation answers four questions: (Q1) does the fault reproduce and does the protocol eliminate it in isolation? (Q2) does each protocol component earn its place? (Q3) does it hold with real LLM re-synthesis on recovery, across models? (Q4) does it survive a production-shaped agent graph with real crashes? All crash injection uses real process kills; tools are atomic idempotent receivers recording every committed effect in an append-only ledger (ground truth). Total API spend was \$0.114. We report *safety* (duplicate-effect rate: ledger commits beyond the first per semantic effect) separately from *completion* (exactly-once rate: every accepted effect committed exactly once). Safety is a protocol property and holds everywhere; completion additionally depends on the recovery planning policy (§9). Denominators are episodes per condition; “duplicate rate” is the fraction of episodes with ≥ 1 duplicate ledger commit.

6.1 Q1: Scripted sandbox

Scripted agents run 4–8-step workflows (read-only plus effectful steps: `issue_refund`, `send_email`,

`update_record`) against mocked tools. The harness process crashes at a random effectful step; the crash lands in-window (after tool commit, before checkpoint write) 75% of the time, else at safe points (pre-step 9.6%, post-checkpoint 14.8% measured). In-memory state is discarded; the agent resumes from the last durable checkpoint. Recovery replanning is nondeterministic (reworded args 50%, plan shift 30%), modeling LLM re-synthesis after restore. Four conditions, 1,500 episodes each; results are in Table 3:

TABLE 3

Sandbox: duplicate-effect and exactly-once rates (1,500 episodes/condition). Zero observed duplicates corresponds to a one-sided exact 95% upper bound of 0.20% on the true duplicate rate.

Condition	Duplicate rate	Exactly-once rate
Baseline (no keys)	0.7687	0.2313
Content-hash keys	0.3667	0.6333
Deterministic keys	0.2113	0.7887
Write-ahead claim log	0.0000	1.0000

The baseline duplicates at essentially the in-window crash rate, as predicted. Content-hash keys halve the damage but still duplicate on reworded retries; deterministic keys do better but still duplicate on plan shifts. The write-ahead log reaches zero duplicates and perfect exactly-once: recovery never depends on the agent’s re-derivation. Zombie probes were fenced 1500/1500 under the log condition.

6.2 Q3: LLM-driven validation

We then put real models in the loop: gpt-4o-mini and gpt-4.1-mini each drive 70-episode campaigns (same crash schedule), with the model performing recovery re-synthesis. The claim discipline is harness-enforced, and the model never sees the raw claim log as a planning input. Results: **0 duplicates** in all 140 episodes. Exactly-once was 66/70 (94.29%) for gpt-4o-mini—the 4 misses were a characterizable model pathology, not a protocol flaw: the model entered re-emission loops, re-firing already-committed claims dozens of times until the harness attempt backstop fired (see §9). A tightened recovery prompt (explicit remaining-claims list handed to the model) eliminated the misses entirely: 70/70 exactly-once, re-emissions falling from 196 to 18, at lower spend (\$0.0188 vs. \$0.0298). gpt-4.1-mini scored 70/70 exactly-once with zero re-emissions and zero plan deviations. Claim adherence was 98.3–100% step-level, 100% episode-level. Table 4 summarizes.

TABLE 4

LLM-driven validation (70 episodes/model, same crash schedule).

Model	Duplicates	Exactly-once	Miss cause
gpt-4o-mini	0	66/70 (94.3%)	re-emission loops
gpt-4o-mini (tight)	0	70/70 (100%)	—
gpt-4.1-mini	0	70/70 (100%)	—

The safety result is prompt-independent; the liveness misses are prompt-fixable. The honest caveat: both models are OpenAI—safety transfers by construction (WAL+fencing), but cross-lab generalization is untested.

6.3 Q4: Production-shaped LangGraph deployment

To answer the “not productionalized” objection by construction, we ported the protocol to a real LangGraph StateGraph agent with real side effects: HTTP POSTs to a separate-process tool server and real file appends, a durable fsync’d claim log, atomic log-fenced checkpoints, and epoch fencing. Crashes are real SIGKILLS delivered by an external watchdog that only observes progress markers; recovery is a new cold-start process. LangGraph’s own checkpoint is deliberately unused—it checkpoints trajectories, the insufficient structure.

v1: linear loop. 180 episodes (60/condition): write-ahead log 0.0000 duplicates / 1.0000 exactly-once; baseline 0.8333 (50/60 in-window episodes duplicated); deterministic keys 0.2333—reproducing the sandbox ordering and magnitudes. 60/60 SIGKILLS delivered, 0 missed, 0 recovery failures; 180/180 zombie probes fenced (90 HTTP + 90 file).

v2: production-shaped graph. A “research assistant” StateGraph with conditional supervisor routing, Send-based parallel fan-out to worker subgraphs with reducer fan-in, and a framework retry loop with real backoff—plus three new crash windows: *mid-fan-out* partial completion (some workers committed, others in flight), *post-branch-decision* pre-tool (a liveness check), and *mid-retry-backoff* (claim exists, uncommitted). Per-branch durable claims, epoch fencing, and log-fenced branch-state throughout. 180 episodes per campaign (60/condition), 0 missed crashes, 0 recovery failures, zombie fencing 60/60 on both tool paths in every condition; the headline numbers are in Table 5:

TABLE 5

LangGraph v2: duplicate / exactly-once rates (60 episodes/condition). Zero observed duplicates corresponds to a one-sided exact 95% upper bound of 4.87% per condition.

Campaign	WAL	Baseline	Deterministic keys
Scripted planner	0.0000 / 1.0000	0.8000 / 0.2000	0.0000 / 1.0000
LLM planner (uniform)	0.0000 / 1.0000	0.8000 / 0.2000	0.0000 / 1.0000
LLM planner (mixed)	0.0000 / 1.0000	0.8000 / 0.2000	0.0000 / 1.0000

Crash-point breakdown (Table 6, mixed campaign): *mid-fan-out* and *retry-backoff* duplicate in baseline (48/48 combined episodes), zero everywhere under WAL/deterministic; *post-branch* duplicates nowhere (the liveness check: all episodes resumed and finished with 0 missing effects). Baseline duplicates arrive 2 per episode (48 episodes $\times$ 2), matching the two unsettled branches.

TABLE 6

LangGraph v2: duplicate episodes by crash point (60 episodes/condition). Crash-point *distributions* are matched across campaigns (scripted/uniform/mixed) by shared seed, not across conditions—within one campaign, the three conditions draw independent crash realizations, so cross-condition comparisons are unpaired; the table reports the mixed campaign.

Condition	mid-fan-out	retry-backoff	post-branch	Total
WAL	0/33	0/20	0/7	0/60
Baseline	30/30	18/18	0/12	48/60
Determ. keys	0/32	0/13	0/15	0/60

The real-LLM supervisor (gpt-4o-mini) decides round-1 routing only and never sees the raw claim log; deter-

ministic reconciliation runs first in both generations. Each LLM campaign runs 180 episodes (60/condition) for 214 round-1 decisions (34 post-branch episodes decide in both generations; cross-generation consistency 34/34). The uniform campaign (temperature 0) degenerated to 214/214 *fan-out*—the prompt showed identical evidence every episode. After a prompt fix (seeded per-episode evidence profiles plus a cost-aware rule), the mixed campaign produced a genuine 135-escalate / 79-*fan_out* split with 0 fallbacks (\$0.0117): the protocol holds under a real mixed routing distribution, not just a uniform one.

Notably, deterministic keys hold at 0.0000 in v2 (unlike v1’s 0.2333) because v2’s branch identities are structural and stable across recovery—an informative contrast sharpening the paper’s claim (§8).

Overhead. Claim-log write latency is the per-step wall-clock cost of the write path: one fsync’d JSONL append per CLAIM record, one per COMMIT record, plus one fsync’d checkpoint-file write per step (~ 376 bytes/record), measured with `perf_counter_ns` in `src/measure_overhead.py` (200 episodes $\times$ 4 steps). Scripted v2 ($n=798$ steps): p50 20.3 ms, p99 63.4 ms; the E2b campaign re-measures p50 21.5 ms, p99 62.7 ms ($n=798$); LLM variant: p50 16.4 ms, p99 54.6 ms. Against multi-second LLM tool calls this is on the order of 1%, but we report the absolute latencies because the ratio depends on the tool-call profile. Log-scale microbenchmark E3b (`src/e3b_log_scale.py`; 100/1,000/10,000 claims): append p50 is flat at ~ 0.03 ms ($O(1)$ append-only discipline holds); `FIND_BY_IDENTITY` is a linear scan at 6.6 ms $\rightarrow$ 16.8 ms $\rightarrow$ 850 ms— $O(n)$, an honest scaling bottleneck: production use needs an index (follow-up work), though all experiments here stay far below the knee. Full log `load()` is 0.79 s at 10,000 claims. Fan-out (E3a: 2/4/8, 30 episodes each) shows 0.0000 duplicates at every fan-out with no per-claim cost change—fan-out multiplies claim count, not per-claim cost; the LLM planner adds no per-claim overhead. Tool-server throughput (E3c, `e3c_loadtest.py`): 688 ops/s at p50 1.3 ms ($K=1$), 636 ops/s at p50 5.4 ms ($K=4$), 310 ops/s at p50 11.2 ms with p99 1,055 ms ($K=16$)—the single-threaded HTTP tool server serializes at high concurrency and would need replacing for production; it does not affect the crash-consistency results. *Paired end-to-end latency* (E5: wal/native/deterministic, 20 episodes each, seed-matched so each episode runs the identical workflow under all three conditions; per-episode wall clock `elapsed_s`; 95% CIs on paired differences, mean $\pm 1.96 \times SE$, $n=20$): wal vs. deterministic $+0.38$ s (95% CI $[-0.08, +0.83]$)—the interval includes zero and extends to $+0.83$ s, so it is inconclusive about a difference and does not establish negligible claim-log overhead (see §6 for the full E5 method); wal vs. native -1.92 s (95% CI $[-2.96, -0.88]$)—the claim-log path is faster than SQLiteSaver checkpoint resume here. *Recovery time* (per-episode `recover_s`, timed around the recovery subprocess): wal mean 5.30 s vs. deterministic 5.43 s (-0.14 s, CI $[-0.53, +0.26]$) and native 6.75 s (-1.46 s, CI $[-2.24, -0.67]$); recovery is dominated by workflow re-execution after the checkpoint, not by log reconciliation. All 60 paired episodes: 0 duplicates, 60/60 exactly-once.

6.4 Boundary experiments: methods and results (E1–E5)

Table 7 summarizes the reviewer-requested boundary experiments: for each, the baseline condition, the identity/key rule under test, the injected fault, and the measured duplicate, missing-effect, and completion counts. All use the v2 LangGraph harness with real SIGKILL unless noted—E4 is the noted exception, a sandbox re-run of the `no_fencing` condition rather than the v2 harness; scoring is against the tool server’s append-only ledger (ground truth), independent of the harness.

E1: native persistence. The v2 agent graph is rebuilt on LangGraph’s `SqliteSaver` with no claim log and no manual recovery node: the fresh generation runs `invoke` with thread state, and on SIGKILL the recovery re-invokes with `None` input and the same thread id, letting the checkpoint resume from the last committed superstep—the framework’s native path. Durable identities are the same deterministic keys as the `det` condition and the receiver contract is unchanged; task boundaries are the framework’s superstep boundaries and key persistence is the checkpoint’s own durable state. Crash schedule and receiver are matched to the WAL condition. Result: 60/60 exactly-once, 0 duplicates.

E2: identity shift. E2a (positional): recovery rotates branch dispatch positions (`b0`→`b2`, `b1`→`b0`, `b2`→`b1`), so positional deterministic keys differ from the pre-crash keys for the same logical effect, and some rotated keys collide with other operations’ committed keys. E2b (content): recovery rewords tool targets through a deterministic invertible paraphrase (`finding-r0-b0` → `finding_r0_b0_rpl`), so content-hash keys differ; scoring canonicalizes targets to the reword-invariant business identity. In both, the WAL recognizes the original identity through the durable claim record—`find_by_identity` on the stored (*action*, *target*, *occurrence*) (E2b additionally canonicalizes the lookup target)—and replays the *original* key, never re-deriving. Result: `det_shift` 46/60 duplicate episodes (0.77 duplicates and 0.77 missing effects per episode: collision and re-derivation modes); `det_content` 46/60 duplicate episodes (92 excess commits, 0 missing); WAL 60/60 in both. E2 evaluates the *combined* design—canonicalized identity lookup plus durable key reuse—against hashing mutable text; it does not isolate the marginal contribution of the log itself, and we do not attribute the entire gap to the log alone. The E1 result remains the boundary: where stable identities survive recovery, keys alone suffice and no new record is needed.

E4: fencing trace. A 200-episode re-run of the sandbox `no_fencing` condition (claim log kept, epoch fencing removed), classifying each duplicate-causing call as (a) delayed same-key recommit vs. (b) surviving writer minting a fresh key at a stale epoch. Result: 102/200 episodes with duplicates; 102/102 offending calls were class (b), 0 class (a). Representative trace (episode 0): the fresh generation commits `send_email` to `customer-1042-s3` under key `det:wf-0776...:3:send_email` at epoch 0; the recovery generation replays the same key at epoch 1 and is correctly suppressed (`duplicate_suppressed`); the surviving zombie—the pre-crash instance, never killed

mid-fan-out and never observing the fence bump—retries the same semantic effect with a fresh content-hash key `ch:33428fa0...` at stale epoch 0, which the receiver accepts because the key is unseen. The receiver’s deduplication is not at fault; the missing piece is fencing the stale generation. This is a *separate fault model* from the main crash-stop evaluation: overlapping generations rather than a clean kill.

E5: paired timing. 20 episodes × 3 conditions (`wal/native/deterministic`), seed-matched (pairing verified: 0/60 mismatches), each with an in-window crash and full recovery. Reported intervals are 95% CIs on the *paired differences* (mean ± 1.96×SE, $n=20$); end-to-end is wall clock from crash through recovery completion, while the recovery row is the instrumented recovery-subprocess interval (reconciliation start through checkpoint advance) within the same episodes. End-to-end: `wal` vs. `deterministic` +0.38s (95% CI [−0.08, +0.83]); `wal` vs. `native` −1.92s (95% CI [−2.96, −0.88]). The `wal`–`deterministic` interval *includes zero and extends to +0.83s*: it is inconclusive about a difference and does *not* establish negligible claim-log overhead—we report the interval, not the claim. Recovery time: `wal` −0.14s vs. `deterministic` (CI [−0.53, +0.26]); `wal` −1.46s vs. `native` (CI [−2.24, −0.67]). All 60 paired episodes: 0 duplicates, 60/60 exactly-once.

6.5 Threats to validity

Construct. Our duplicate metric counts ledger commits beyond the first per semantic effect; the ledger is the tool server’s own append-only record, independent of the harness. **Internal.** Crash timing is seeded and recovery non-determinism (reword/shift rates) is modeled rather than observed in the wild; different rates would shift the keys-alone numbers but not the WAL result, which is independent of re-derivation by construction (Theorem 2). **External.** All crashes are single-crash, crash-stop, and harness-side; multi-crash episodes, tool-side crashes, and Byzantine harnesses are out of scope. Both validated LLMs are from one vendor, and the production port uses one framework (LangGraph); the mechanism is framework-agnostic by design (harness-enforced claims, tool-side idempotent receivers), but other frameworks are untested.

7 ABLATION STUDY

Every protocol piece must earn its place. A 1,500 episodes/condition component ablation adds a second, *explicitly separate* fault model—the *zombie writer* described in E4 above: with probability 0.5, the pre-crash process instance is *not* killed and retries its in-flight step with its stale epoch, minting a fresh content-hash key the new generation never saw. The tool’s idempotent receiver cannot suppress it (unseen key); only the epoch fence rejects it. This is overlapping generations, not the crash-stop fault of §5; we evaluate it separately because production kills are not always clean and fencing exists precisely for the unclean case. Table 8 gives the results:

Removing fencing regresses to 0.4927 (zombie commits land); removing the claim log regresses to 0.2333—reproducing the deterministic-keys number, confirming the

TABLE 7

Boundary experiments: baseline, identity/key rule, injected fault, and outcomes. “det” = deterministic keys re-derived at retry time; “wal” = write-ahead claim log. Duplicates = ledger commits beyond the first per semantic effect; missing = intended effects never committed; completion = episodes with every accepted effect committed exactly once.

Exp.	Baseline	Identity/key rule	Injected fault	Eps	Dup. eps	Missing/ep	Completion
E1	native (Sqlite-Saver)	deterministic keys, no claim log; checkpoint resume from last superstep	SIGKILL at retry_backoff / mid_fanout / post_branch (26/16/18)	60	0	0.00	60/60
E2a	det_shift	positional keys (<i>wid, i, action</i>); recovery rotates branch indices	plan-shift replan on recovery	60	46	0.77	14/60
E2a	wal	claim-log identities; <i>find_by_identity</i> key reuse	same	60	0	0.00	60/60
E2b	det_content	content-hash keys <i>H(action, args)</i> ; recovery rewords targets	invertible paraphrase on recovery	60	46	0.00	14/60
E2b	wal	claim-log identities; canonicalized lookup claim log	same	60	0	0.00	60/60
E3	wal		fan-out $\in \{2, 4, 8\}$; log-length $\{10^2, 10^3, 10^4\}$; concurrency $K \in \{1, 4, 16\}$	90+	0	0.00	all
E4	no_fencing	claim log kept, epoch fencing removed	zombie writer (surviving pre-crash process, stale epoch)	200	102	—	—
E5	wal/det/native	paired seed-matched episodes	none (paired timing; episodes include crash + recovery)	60	0	0.00	60/60

TABLE 8

Component ablation: duplicate-effect and exactly-once rates (1,500 episodes/condition, zombie-writer fault model); zero observed duplicates $\rightarrow$ one-sided exact 95% upper bound 0.20%.

Condition	Duplicate rate	Exactly-once
Full protocol	0.0000	1.0000
No fencing	0.4927	0.5073
No claim log	0.2333	0.7667
Neither	0.8767	0.1233

log is what defeats key re-derivation; removing both reaches 0.8767 (the window component plus the zombie component). The components are load-bearing against *disjoint* failure modes: the log defeats key re-derivation, the fence defeats stale generations; neither subsumes the other. Zombie probes were fenced $\approx 750/1500$ wherever fencing was on, 0 fenced (and ≈ 740 committed as duplicates) wherever removed.

8 DISCUSSION: THE KEY-DERIVATION BOUNDARY

Our results draw a sharp empirical boundary around when idempotency keys suffice. In the v1 linear loop, deterministic (*workflow, step, action*) keys duplicate at 0.2333: recovery replanning shifts step indices (30% of episodes), so the retry re-derives a different key. In the v2 branching graph,

the *same* key scheme holds at 0.0000: branch identities are structural and stable across recovery, so re-derivation coincides. Keys fail *exactly when* post-recovery recomputation re-derives different identities—and for LLM agents, whether identities survive recovery is not a property the harness can assume. It depends on the agent’s re-synthesis behavior, the plan structure, and the retry path: precisely the things that vary across deployments and model versions.

This is why Theorem 1 matters beyond the formalism: the boundary is not an artifact of our two key schemes but a property of *retry-time derivations from mutable inputs* under unconstrained re-synthesis. A practitioner reading our v2 result (“deterministic keys work here”) might conclude keys suffice; E2a/E2b and the theorem together say that conclusion does not transfer across the identity-stability boundary. The write-ahead claim log is the mechanism that makes recovery independent of which side of the boundary a given deployment lands on. The protocol is also *planner-agnostic* in practice: scripted, uniform-LLM, and mixed-LLM supervisors produce headline-identical numbers in all conditions, as the fault model predicts—the fault class is harness-side, and the mechanism composes with arbitrary agent cognition rather than depending on it.

9 LIMITATIONS

We state the limits plainly; they are what make the safety claims credible. **Model re-emission loops:** gpt-4o-mini’s 4

misses (94.29% exactly-once) came from the model re-firing committed claims dozens of times until the harness back-stop fired—a liveness pathology, never a safety violation (0 duplicates throughout). A tighter recovery prompt reached 100%, so the limitation is prompt-fixable, but re-emission-aware recovery prompting remains future work. **Single vendor:** both validated models are OpenAI; safety transfers by construction (WAL+fencing), but cross-lab generalization is untested. **Uniform-route limitation and fix:** the first LLM-supervisor campaign degenerated to 214/214 *fan_out* decisions (180 episodes) because the prompt showed identical evidence every episode; the fixed prompt (seeded evidence profiles + cost-aware rule) produced the genuine 135/79 mixed split under which the protocol still holds—we report both, because the failure mode (prompts without a real decision are defaults, not judgments) generalizes. **Scope:** crash-stop harness only, one crash per episode, tool-side crashes out of scope; the tool idealization (L1) and log-durability assumption (L2) are stated in §5. **Evaluation scale:** 1,500 episodes/condition in simulation and 60/condition in the production port reflect a fault-injection budget, not a production trace; the crash distributions are seeded and the recovery nondeterminism is modeled, not observed in the wild. **Smarter compression:** where recovery must compress rather than replay history, compressor quality bounds fidelity—a follow-up direction, not a claim of this paper. **Original-result recovery:** the receiver contract returns `DUPLICATE_SUPPRESSED` status only, not the original result payload (RIFL returns the saved result; we do not). A downstream step that needs the pre-crash call’s return value must re-derive it or tolerate its absence; extending the claim record to carry result payloads (with the attendant storage and privacy costs) is specified future work. **Concurrent recoveries:** the single-recovery-writer assumption (A7) is load-bearing—epoch acquisition is read-increment-write on the harness side and atomic only via T ’s *max*-monotonic fence, so two concurrent recoveries of the same workflow could share an epoch and mutually admit stale writes. Fencing makes this a deployment-operations constraint (one recovery at a time), not a protocol guarantee; a distributed fence (lease, consensus) would be needed to lift it.

10 CONCLUSION

Agent harnesses checkpoint trajectories; the world commits effects; the gap between them is a crash window that duplicates side effects at rates of 0.77–0.83 in our measurements. Idempotency keys narrow the gap but cannot close it in general—we prove (Theorem 1) that when a retry-time key derivation actually changes across an admissible recovery replan of the same logical effect, a keys-only receiver can commit both attempts, and that shifted keys colliding with another operation’s committed key cause erroneous suppression instead. The write-ahead effect-claim log closes the *safety* gap: durable claims before invocation, log-fenced checkpoints, reconciliation-first recovery, and epoch fencing commit zero duplicate effects across sandbox, two-model LLM validation, component ablation, and production-shaped LangGraph campaigns with real SIGKILL injection, at measured claim-log write latency of p50 20.3 ms per step. Workflow *completion* reaches

1.000 everywhere except gpt-4o-mini under the loose recovery prompt (66/70; prompt-fixable re-emission loops, tight prompt: 70/70)—a liveness property of the planning policy, not of the commit protocol. The log is not an optimization; for LLM agents, durable write-ahead identity information is the minimal requirement that makes duplicate-free recovery possible under our recovery-information model, and our claim log is a sufficient mechanism.

REFERENCES

- [1] J. Li, “Where does exactly-once live? Model, harness, and tool-contract effects on duplicate side effects in LLM agents,” arXiv:2609.29095, Sep. 2026.
- [2] Y. Zheng, Y. Yang, W. Zhang, and A. Quinn, “ACRFence: Preventing semantic rollback attacks in agent checkpoint-restore,” arXiv:2603.20625, Mar. 2026.
- [3] Y. Zhuang, K. Chen, Y. Duan, S. Zheng, J. Li, and X.-Y. Zhang, “AgentRewind: Recoverable execution for long-horizon LLM agents,” arXiv:2608.14380, 2026.
- [4] S. Khan, “Resume means resume: A machine-checked conformance contract for checkpoint, interrupt, and resume semantics in workflow persistence layers,” arXiv:2608.03836, Aug. 2026.
- [5] Y. Zheng, X. Song, Y. Hu, L. Cheng, Y. Huang, and W. Zhang, “When can agents safely checkpoint, fork, restore, and merge? Exact checking for execution edits,” arXiv:2608.22928, Aug. 2026.
- [6] G. Wu, D. Li, K. Jiang, J. Niu, C. Wang, and Y. Zhang, “Safe to resume? Breaking execution continuity of agent execution via rollback,” arXiv:2608.29381, Aug. 2026.
- [7] M. Balakrishnan *et al.*, “LogAct: Enabling agentic reliability via shared logs,” arXiv:2604.07988, Apr. 2026.
- [8] LangChain, “LangGraph: Stateful multi-actor applications with LLMs,” <https://github.com/langchain-ai/langgraph>, 2026.
- [9] C. Lee, S. J. Park, A. Kejriwal, S. Matsushita, and J. Ousterhout, “Implementing linearizability at large scale and low latency,” in *Proc. 25th ACM Symp. Operating Systems Principles (SOSP’15)*, Monterey, CA, 2015, pp. 71–86.
- [10] C. Mohan, D. Haderle, B. Lindsay, H. Pirahesh, and P. Schwarz, “ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging,” *ACM Trans. Database Syst.*, vol. 17, no. 1, pp. 94–162, Mar. 1992.
- [11] H. Garcia-Molina and K. Salem, “Sagas,” in *Proc. ACM SIGMOD Int. Conf. Management of Data*, San Francisco, CA, 1987, pp. 249–259.